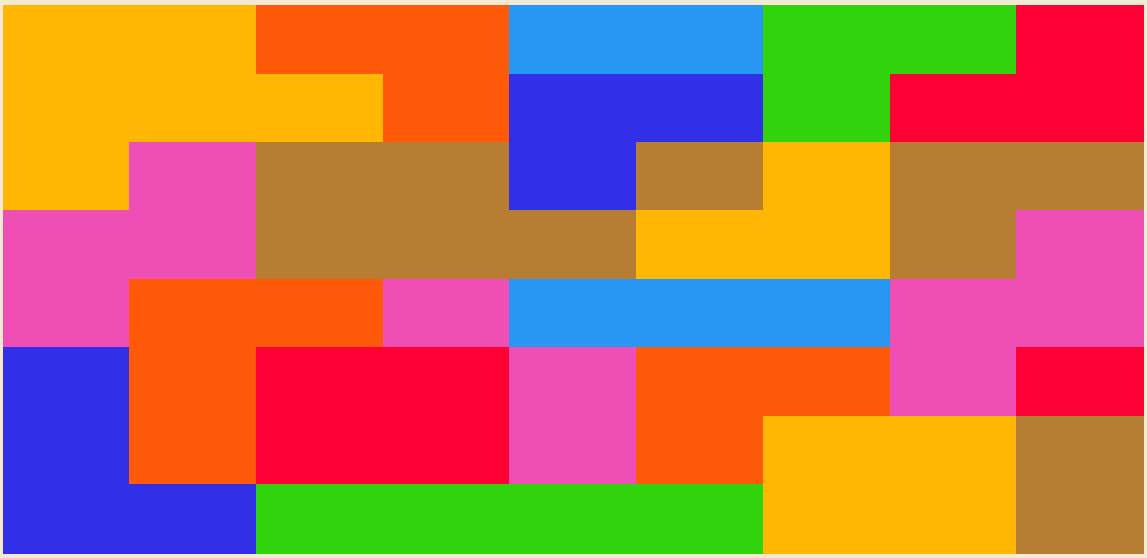

DRONE CAMP

DAY 3

Loops, decisions, and reusable flight routines.

PYTHON + EASYTELLO + CONTROL FLOW

MISSION

Make the flight plan think

On Day 2, a Python file described one fixed sequence of EasyTello commands. Day 3 adds control flow: rules that decide which statements execute, how many times they execute, and which named routine should run. Python still reads from top to bottom, but a loop can return to an earlier block, a conditional can skip one branch and choose another, and a function can package several commands under one useful name.

The central question	Which part of the program controls repetition, which part controls decisions, and which part gives a flight routine a reusable name?

Three tools, three jobs

- for loop: repeat a known pattern a known number of times
- conditional: choose a branch after testing a true-or-false expression
- function: group related instructions and allow values to enter through parameters

PROGRAM MAP

```
Python control flow
        |
        +-- for loop -> repeat commands
        |
        +-- if/elif/else -> choose a route
        |
        `-- function -> name and reuse a routine
                 |
                 v
            EasyTello methods
                 |
                 v
            Tello command messages
```

Python decides what should happen. EasyTello translates the selected method calls into commands that the drone understands.

CHECKPOINT 01

Repeat a movement pattern with a for loop

FILE LEVEL - A FOUR-SIDE ROUTE

```
from easytello import tello

drone = tello.Tello()

drone.takeoff()
drone.up(30)

for side in range(4):
    drone.forward(40)
    drone.cw(90)
    drone.wait(1)

drone.land()
drone.close()
```

What range(4) contributes

range(4) provides four loop cycles. During those cycles, side receives 0, then 1, then 2, then 3. The program does not need to display those values; Python still uses them to count the repetitions. The indented block belongs to the loop, so all three method calls repeat together.

Why this pattern suggests a square

Each cycle moves forward once and rotates ninety degrees once. Repeating that pair four times creates four sides and four turns. Four ninety-degree turns total 360 degrees, so the intended final heading matches the starting heading. Real flight may drift slightly, which is why the route should be short and the flight area should remain clear.

Trace first	Write the expected position and heading after each cycle before the propellers begin moving.

CHECKPOINT 02

Choose a branch with a conditional

FILE LEVEL - IF/ELSE CHOOSES WHETHER TO FLY

```
from easytello import tello

MIN_BATTERY = 40

drone = tello.Tello()
battery = int(drone.get_battery())

print(f"Battery: {battery}%")

if battery < MIN_BATTERY:
    print("Battery is below the lesson threshold.")
    print("Charge the drone before flying.")
else:
    print("Battery check passed.")
    drone.takeoff()
    drone.wait(2)
    drone.land()

drone.close()
```

The expression becomes True or False

The condition battery < MIN_BATTERY compares two integers. Python evaluates that comparison before choosing a branch. When the expression is True, only the first indented block runs. When it is False, Python skips that block and runs the else block instead. The branches are mutually exclusive during one execution.

Why convert the response with int()?

get_battery() returns the drone's response. int(...) converts a numeric response into an integer so Python can compare it with 40. The threshold here is a classroom rule for this exercise, not a replacement for the drone's official operating guidance or an instructor's preflight check.

Decision point	A conditional does not move the drone by itself. It decides which EasyTello method calls Python will reach.

CHECKPOINT 03

Give a flight routine a function name

FUNCTION DEFINITION -> FUNCTION CALL

```
def fly_square(drone, distance, angle):
    drone.takeoff()
    drone.up(30)

    for side in range(4):
        drone.forward(distance)
        drone.cw(angle)
        drone.wait(1)

    drone.land()


from easytello import tello

my_drone = tello.Tello()
fly_square(my_drone, 40, 90)
my_drone.close()
```

Definition and call are different moments

The def statement teaches Python what fly_square means. The indented body is stored as a reusable routine, but it does not execute at the moment Python reads the definition. The later function call starts the routine and supplies three arguments: the drone object, a distance, and an angle.

Parameters make the routine adjustable

Inside the function, drone, distance, and angle are parameters. They receive the arguments supplied by the call. Because the routine uses those names instead of fixed values, another call can reuse the same logic with a different distance or turn angle. The algorithm remains recognizable while its inputs change.

Read it aloud	Define a routine named fly_square that needs a drone, a distance, and an angle.

CHECKPOINT 04

Map decisions to named flight routines

FUNCTIONS DEFINE ROUTES - CONDITIONAL SELECTS ONE

```
def fly_line(drone, distance):
    drone.takeoff()
    drone.up(30)
    drone.forward(distance)
    drone.wait(1)
    drone.land()


def fly_square(drone, distance):
    drone.takeoff()
    drone.up(30)

    for side in range(4):
        drone.forward(distance)
        drone.cw(90)
        drone.wait(1)

    drone.land()


if battery >= 60:
    fly_square(drone, 40)
elif battery >= 40:
    fly_line(drone, 40)
else:
    print("Charge the drone before flying.")
```

How if, elif, and else are tested

Python checks the conditions from top to bottom. A battery value of 72 satisfies the first comparison, so fly_square runs and the remaining branches are skipped. A value of 48 fails the first comparison but satisfies the elif comparison, so fly_line runs. A lower value reaches the else branch. Exactly one branch is selected.

Why functions belong at the branch endpoints

The conditional remains readable because each branch names a complete behavior instead of containing every movement command directly. The decision answers which route should run; the selected function answers how that route works. Separating those responsibilities makes both the logic and the movement sequence easier to inspect.

Design habit	Use conditionals to choose behavior and functions to describe behavior.

FINAL BUILD

Combine loops, decisions, functions, and EasyTello

COMPLETE DAY 3 PROGRAM

```
from easytello import tello

MIN_BATTERY = 40
MOVE_DISTANCE = 40
TURN_ANGLE = 90


def battery_ready(drone):
    battery = int(drone.get_battery())
    print(f"Battery: {battery}%")
    return battery >= MIN_BATTERY


def fly_square(drone, distance, angle):
    drone.takeoff()
    drone.up(30)

    for side in range(4):
        drone.forward(distance)
        drone.cw(angle)
        drone.wait(1)

    drone.land()


def run_mission():
    drone = tello.Tello()

    if battery_ready(drone):
        fly_square(
            drone,
            MOVE_DISTANCE,
            TURN_ANGLE
        )
    else:
        print("Charge the drone before flying.")

    drone.close()


run_mission()
```

Follow the values through the program

run_mission() creates the EasyTello object. battery_ready(drone) requests a value and returns True or False. The if statement uses that return value to choose whether fly_square(...) runs. Inside the selected routine, the loop repeats the forward-turn pattern. Every movement still reaches the drone through an EasyTello method.

Before launch	Trace the call order: run_mission -> battery_ready -> conditional -> fly_square -> EasyTello methods.

RECAP

What Python controls and what EasyTello controls

Python's control-flow tools arrange the mission. A for loop determines how often a block repeats. A conditional determines which branch executes. A function gives a multi-command routine a name and accepts values through parameters. EasyTello handles a different responsibility: each reached method constructs and sends a command such as forward 40 or cw 90, then records the drone's response.

FIVE QUESTIONS FOR READING A FLIGHT PROGRAM

```
for loop      -> how many repetitions?
conditional   -> which branch?
function      -> which named routine?
argument      -> which distance or angle?
EasyTello     -> which command reaches the drone?
```

Common mistakes

- Loop body is not indented, so only part of the route repeats.
- Function is defined but never called.
- Argument order does not match the parameter order.
- Condition uses text instead of an integer battery value.
- Several routes each contain takeoff and land, then more than one route is called.
- The file is started before the computer connects to the Tello Wi-Fi network.

Safe testing sequence

Connect to the Tello network, confirm the battery response, trace the route on paper, clear the flight area, and test the smallest complete routine first. Keep one operator responsible for launch. Use emergency() only for a real emergency because it stops the motors immediately rather than performing a normal landing.

Final understanding check	Explain how changing range(4) to range(2) differs from changing MOVE_DISTANCE from 40 to 60.